tiling模板化编程、属性、Tbuf、workspace的使用

算子开发中，当需要实现多逻辑适配、带属性算子开发、运算过程中间数据缓存时，需重点掌握TilingKey属性TBuf、workspace的使用技巧。 本次学习将围绕以下框架展开，带你逐一掌握tiling模板化编程、属性、TBuf、workspace 的应用方法：

- 环境准备
- workspace介绍
- Tbuf介绍
- 什么是属性
- 什么TilingKey
- 开发使用workspace、属性、Tilingkey的算子
- 课后实践

---

1. 环境准备

本文所有内容均存放于Sources文件夹。 在开始创建算子工程前，先要对jupyter环境进行初始化。以下代码完成了初始化并将环境中的变量导入jupyter环境，并完成代码目录的创建。保证能正常导入代码以及代码能编译运行。

In [ ]:
```
!mkdir -p Sources/03.05

import os, subprocess
env = subprocess.check_output("bash -l -c 'source $ASCEND_TOOLKIT_HOME/set_env.sh &
for line in env.splitlines():
    if "=" in line: os.environ.__setitem__(*line.split("=", 1))
print("\n🎉 Environment initialization process completed successfully!")
```

验证 msopgen 工具（CANN 自带）

In [ ]:
```
# CANN 9.0.0 已自带 msopgen 工具，无需源码编译安装
!which msopgen && msopgen -h
```

---

2. workspace介绍

2.1 什么是workspace

workspace是设备侧Global Memory上的一块内存。workspace内存分为两部分：系统workspace和用户workspace。

- 系统workspace：Ascend C API需要预留的workspace内存
  API在计算过程需要一些workspace内存作为缓存，因此算子需要为API预留workspace内存，预留内存大小通过GetLibApiWorkSpaceSize接口获取。

- 用户workspace：算子实现使用到的workspace内存
  算子内部需要通过额外的device内存进行数据交换或者缓存的时候才需要分配，根据实际情况自行分配。使用场景如下：

  - 需要使用Unified Buffer和L1 Buffer上的空间且空间不够用时，可以将数据暂存至workspace上。
  - 调用SyncAll等API接口时，需要workspace作为入参。
  - 其他需要使用Global Memory上内存空间的场景。

   turing_test

2.2 如何使用workspace

在算子工程tiling函数中先通过GetWorkspaceSizes接口获取workspace大小的存放位置，再设置workspace的大小，框架侧会为其申请对应大小的设备侧Global Memory，在对应的算子kernel侧实现时可以使用这块workspace内存。在使用Matmul Kernel侧接口等需要系统workspace的高阶API时，设置的workspace空间大小为系统workspace和用户workspace之和。例如：

```
// 用户自定义的tiling函数
static ge::graphStatus TilingFunc(gert::TilingContext* context)
{
    AddApiTiling tiling;
    ...
    size_t usrSize = 256; // 设置用户需要使用的workspace大小为256字节。
    // 如需要使用系统workspace需要调用GetLibApiWorkSpaceSize获取系统workspace的大小。
    auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformInfo());
    uint32_t sysWorkspaceSize = ascendcPlatform.GetLibApiWorkSpaceSize();
    size_t *currentWorkspace = context->GetWorkspaceSizes(1); // 通过框架获取workspace的指针，GetWorkspaceSizes入参为所需workspace的块数。当前限制使用一块。
    currentWorkspace[0] = usrSize + sysWorkspaceSize; // 设置总的workspace的数值大小，总的workspace空间由框架来申请并管理。
    ...
}
```

在算子kernel侧实现时，即可使用这块workspace内存。使用方法核输入输出类似，例如：

```
// 用户自定义的kernel函数
template <typename D_T_X, typename D_T_Y>
 __global__ __aicore__ void clamp(GM_ADDR x, GM_ADDR y, GM_ADDR workspace, GM_ADDR tiling) {
  AscendC::GlobalTensor<float> tmpGm;
  tmpGm.SetGlobalBuffer((__gm__ float *)workspace);  // 初始化
```

```
GlobalTensor，将workspace内存作为GlobalTensor的内存
 }
```

---

3. TBuf介绍

3.1 TBuf的作用

上文我们提到workspace可以用来暂存数据，但是由于workspace使用的是GlobalMemory上的内存，使用workspace暂存数据时需要做搬出操作，读取workspace内暂存的数据时需要做搬入操作，会导致算子性能变差。在大多数算子开发时，我们使用另外一种使用LocalMemory的TBuf进行暂存数据。

TBuf主要是用于Vector计算申请的临时空间，不需要有释放动作。TBuf和TQue 相同和不同如下：

相同： TBuf和TQue都通过InitBuffer来初始化内存

不同：

- 获取内存，TBuf通过Get()，TQue通过AllocTensor()
- TBuf分配的内存空间只参与计算，无法执行入队出队操作。TQue的出入队列EnQue和DeQue必须成对出现。
- TBuf申请的内存无需释放，TQue申请内存AllocTensor和释放内存FreeTensor必须成对出现，

3.2 如何使用TBuf

这里我们以实现bfloat16类型计算的Add为例，介绍一下使用TBuf的关键代码，代码整体流程如下：
 turing_test

与基础矢量算子实现的KernelAdd算子类相比，算子类新增三个TBuf类型的成员变量tmpBuf0、tmpBuf1、tmpBuf2，用于管理计算过程中使用的临时内存，代码如下。

```
class KernelAdd {
public:
    __aicore__ inline KernelAdd() {}
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z){}
    __aicore__ inline void Process(){}
private:
    __aicore__ inline void CopyIn(){}
    __aicore__ inline void Compute(){}
    __aicore__ inline void CopyOut(){}
private:
```

```
    AscendC::TPipe pipe;
    AscendC::TQue<AscendC::TPosition::VECIN, 1> inQueueX, inQueueY;
    AscendC::TQue<AscendC::TPosition::VECOUT, 1> outQueueZ;
    // 新增三个TBuf类型的成员变量
    AscendC::TBuf<AscendC::TPosition::VECCALC> tmpBuf0, tmpBuf1, tmpBuf2;
    AscendC::GlobalTensor<bfloat16_t> xGm;
    AscendC::GlobalTensor<bfloat16_t> yGm;
    AscendC::GlobalTensor<bfloat16_t> zGm;
};
```

初始化函数阶段除原有步骤外，需要调用InitBuffer接口为TBuf变量分配内存，具体的初始化函数代码如下：

```
__aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR z)
{
    xGm.SetGlobalBuffer((__gm__ half *)x, TOTAL_LENGTH);
    yGm.SetGlobalBuffer((__gm__ half *)y, TOTAL_LENGTH);
    zGm.SetGlobalBuffer((__gm__ half *)z, TOTAL_LENGTH);

    pipe.InitBuffer(inQueueX, 1, TOTAL_LENGTH * sizeof(bfloat16_t));
    pipe.InitBuffer(inQueueY, 1, TOTAL_LENGTH * sizeof(bfloat16_t));
    pipe.InitBuffer(outQueueZ, 1, TOTAL_LENGTH * sizeof(bfloat16_t));
    // 调用InitBuffer接口为TBuf变量分配内存
    pipe.InitBuffer(tmpBuf0, TOTAL_LENGTH * sizeof(float));
    pipe.InitBuffer(tmpBuf1, TOTAL_LENGTH * sizeof(float));
    pipe.InitBuffer(tmpBuf2, TOTAL_LENGTH * sizeof(float));
}
```

Compute函数使用Tbuf.Get获取暂存数据的Tensor，并使用Tensor参与运算。**注意不要对TBuf.Get获取的Tensor进行EnQue、DeQue或者FreeTensor操作**

```
__aicore__ inline void Compute()
{
    AscendC::LocalTensor<bfloat16_t> xLocal = inQueueX.DeQue<bfloat16_t>
();
    AscendC::LocalTensor<bfloat16_t> yLocal = inQueueY.DeQue<bfloat16_t>
();
    AscendC::LocalTensor<bfloat16_t> zLocal =
outQueueZ.AllocTensor<bfloat16_t> ();
    // 获取暂存数据的Tensor
    AscendC::LocalTensor<float> tmpTensor0 = tmpBuf0.Get<float>();
    AscendC::LocalTensor<float> tmpTensor1 = tmpBuf1.Get<float>();
    AscendC::LocalTensor<float> tmpTensor2 = tmpBuf2.Get<float>();
    AscendC::Cast(tmpTensor0, xLocal, AscendC::RoundMode::CAST_NONE,
TOTAL_LENGTH);
    AscendC::Cast(tmpTensor1, yLocal, AscendC::RoundMode::CAST_NONE,
TOTAL_LENGTH);
    AscendC::Add(tmpTensor2, tmpTensor0, tmpTensor1, TOTAL_LENGTH);
    // 运算完成后，把数据放入结果Tensor中，不要对TBuf的Tensor进行搬运或入队、出
队、释放操作
    AscendC::Cast(zLocal, tmpTensor2, AscendC::RoundMode::CAST_RINT,
TOTAL_LENGTH);
```

```
    outQueueZ.EnQue<bfloat16_t>(zLocal);
    inQueueX.FreeTensor(xLocal);
    inQueueY.FreeTensor(yLocal);
}
```

---

4. 什么是 Ascend C 算子属性

Ascend C 算子属性是算子定义阶段即确定、计算过程中保持不变的**静态参数**，作为算子的固有配置，它与动态变化的输入张量相互独立，核心作用是约束和控制算子的计算行为。



4.1 必选属性

必选属性是算子运行的核心前提，若未指定则算子无法完成初始化或执行，直接定义了算子**要做什么**，是算子存在的必要条件。

- PyTorch示例：调用 `Conv2d` 算子时，必须传入 `in_channels`（输入通道数）、`out_channels`（输出通道数）、`kernel_size`（卷积核尺寸）—— 缺少输入通道数，卷积层无法确定权重矩阵形状；缺少输出通道数，卷积计算的核心逻辑便无从展开。
- Ascend C 对应逻辑：如开发ELU 算子时，`alpha` 可作为必选属性（若未指定，负数部分的缩放逻辑无法确定）；开发卷积类算子时，输入/输出通道数、卷积核尺寸同样是必选属性，无这些参数则无法完成算子核心逻辑的初始化。

4.2 可选属性

可选属性是算子的辅助配置，若未显式指定则使用默认值，不改变算子的核心功能，仅调整计算细节，控制算子**“怎么做”**。

- PyTorch示例：`Conv2d` 的 `stride`（步长）、`padding`（填充）、`bias`（偏置）等属性，默认值分别为 1、0、True，不改变“执行卷积运算”的核心事实，仅调整输出特征图尺寸、边缘保留策略等细节。
- Ascend C 对应逻辑：如开发GELU 算子的 `approximate`（近似计算方式）、Clamp算子的 `min`（设置数值的下限）均可作为可选属性，未指定时使用默认值（如 `approximate="none"`、`min=0`），仅调整计算精度或设置数值的下限为0，不改变算子的核心计算目标。

4.3 核心总结

Ascend C 算子属性与 PyTorch 算子属性的本质完全一致：

1. 静态性：运行过程中不可修改，需在 Host 侧调用算子前确定；

2. 解耦性：与输入张量解耦，不依赖数据的形状/数值，仅控制计算逻辑；
3. 分类逻辑：通过“必选/可选”划分，必选属性决定算子核心功能，可选属性定制计算细节。

4.4 属性使用

在创建算子原型IR时，如果我们想添加一个属性，则应在json内添加attr字段，并在里面配置所需要的属性。例如我们想给之前课程的AddCustomTemplate算子添加float类型的max、min属性用于设置Add计算的最大值和最小值，可按照如下设置:

```
[{
    "op": "AddCustomTemplate",
    "input_desc": [{
            "name": "x",
            "param_type": "required",
            "format": ["ND", "ND"],
            "type": ["float16", "float"]
        },
        {
            "name": "y",
            "param_type": "required",
            "format": ["ND", "ND"],
            "type": ["float16", "float"]
        }
    ],
    "output_desc": [{
        "name": "z",
        "param_type": "required",
        "format": ["ND", "ND"],
        "type": ["float16", "float"]
    }],
    "attr":[
        {
            "name":"max",
            "type":"float",
            "param_type": "optional",
            "default_value":0
        },
        {
            "name":"min",
            "type":"float",
            "param_type": "optional",
            "default_value":0
        }
    ]
}]
```

带属性的算子两段式接口，其第一段通常为 aclnn算子名GetWorkspaceSize(输入1/输入2....., 属性1/属性2....., 输出1/输出2....., workspaceSize, executor) 形式。调用时需传入属

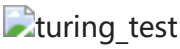
性值；若无需指定属性，需传入默认属性值，确保参数个数与接口定义一致。

---

5. Tiling模板编程介绍

5.1 什么是TilingKey

TilingKey是一个算子内为了区分不同的实现而将kernel代码进行区分的方法，该方法类似于C++的Template模板机制，编译时会根据不同tilingkey会形成不同二进制算子om文件，有助于优化单次调用kernel的性能。



不同的kernel实现分支可以通过TilingKey来标识，host侧设置TilingKey后，可以选择对应的分支。例如我们在Tiling实现函数内设置TilingKey：

```
static ge::graphStatus TilingFunc(gert::TilingContext* context)
{
    // some code
    if (condition) {
        context->SetTilingKey(1);
    } else {
        context->SetTilingKey(2);
    }
    return ge::GRAPH_SUCCESS;
}
```

然后在核函数内编写对应的处理分支：

```
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x, GM_ADDR y,
GM_ADDR z, GM_ADDR workspace, GM_ADDR tiling)
{
    if (TILING_KEY_IS(1)) {
        ProcessA();
    } else if (TILING_KEY_IS(2)) {
        ProcessB();
    }
}
```

5.2 如何使用Tiling模板编程

TilingKey不易于记忆和理解，因为它们通常是较长又没有明确含义的数字。
在涉及多个TilingKey的场景中，开发者依赖TilingKey来管理kernel的实现，无论是在管理还是使用上都会遇到相当大的复杂性。为了简化这一过程，**可以采用模板编程的方法来替代传统的TilingKey编程，从而减少对TilingKey数值标识的依赖，使kernel的管理更加直观和高效。**

在使用Tiling模板编程前，我们需要先了解模板参数常见宏的作用，参考模板参数定义。

我们即可在算子工程op_kernel目录下创建一个模板定位文件，以AddCustomTemplate为例，可以创建一个名为tiling_key_add_custom_template.h的文件（注意：该文件名可自定义，推荐使用tiling_key_+算子名.h的形式命名）。在模板文件内我们需要先引用固定的头文件：

```
#include "ascendc/host_api/tiling/template_argument.h"
```

然后可以定义自己所需要的模板参数，比如我们之前课程实现的AddCustomTemplate支持half和float两种类型数据的计算, 并且核内切分1或8次，那么我们可以定义模板为：

```
ASCENDC_TPL_ARGS_DECL(AddTemplateCustom, // 算子OpType
ASCENDC_TPL_DATATYPE_DECL(D_T_X, C_DT_FLOAT, C_DT_FLOAT16,
ASCENDC_TPL_INPUT(0)),  // DataType类型的模板参数定义：输入参数x的数据类型，取值范围为float16/float32, ASCENDC_TPL_INPUT(0)说明对应Kernel侧第0个输入
ASCENDC_TPL_DATATYPE_DECL(D_T_Y, C_DT_FLOAT, C_DT_FLOAT16,
ASCENDC_TPL_INPUT(1)),  // DataType类型的模板参数定义：输入参数y的数据类型，取值范围为float16/float32, ASCENDC_TPL_INPUT(1)说明对应Kernel侧第1个输入
ASCENDC_TPL_DATATYPE_DECL(D_T_Z, C_DT_FLOAT, C_DT_FLOAT16,
ASCENDC_TPL_OUTPUT(0)), // DataType类型的模板参数定义：输入参数z的数据类型，取值范围为float16/float32, ASCENDC_TPL_OUTPUT(0)说明对应Kernel侧第0个输出
ASCENDC_TPL_UINT_DECL(TILE_NUM, ASCENDC_TPL_8_BW, ASCENDC_TPL_UI_MIX, 2,
0, 2, 3, 5, 10, 12, 13, 9, 8),// 自定义UINT类型（无符号整形）的模板参数定义：模板参数为切分的块数，编码位宽为ASCENDC_TPL_8_BW即8比特，表示该模板参数的个数不超过8比特能表达的范围；ASCENDC_TPL_UI_MIX表示通过混合模式表达取值范围，有2组的数据{0-2}、{3-5}和穷举值10、12、13、9、8，最后结果为{0, 1, 2, 3, 4, 5, 10, 12, 13, 9, 8}
ASCENDC_TPL_BOOL_DECL(IS_SPLIT, 0, 1), // 自定义bool类型的模板参数定义：模板参数为是否切分标志位，取值范围为0和1，1表示切分，0表示不切分
);
```

host侧调用ASCENDC_TPL_SEL_PARAM接口自动生成并配置TilingKey：

```
#include "register/op_def_registry.h"
#include "add_template_custom_tiling.h"
#include "../op_kernel/tiling_key_add_template_custom.h"

namespace optiling {
const uint32_t NUM_BLOCKS = 8;
const uint32_t DEFAULT_TILE_NUM = 8;
constexpr int MIN_LENGTH_FOR_SPLIT = 2048;
static ge::graphStatus TilingFunc(gert::TilingContext *context)
{
    TilingData tiling;
    uint32_t totalLength = context->GetInputShape(0)->GetOriginShape().GetShapeSize();
    ge::DataType dtype_x = context->GetInputDesc(0)->GetDataType();
    ge::DataType dtype_y = context->GetInputDesc(1)->GetDataType();
    ge::DataType dtype_z = context->GetOutputDesc(1)->GetDataType();
    uint32_t D_T_X = static_cast<int>(dtype_x), D_T_Y = static_cast<int>(dtype_y), D_T_Z = static_cast<int>(dtype_z), TILE_NUM = 1, IS_SPLIT = 0;
    if(totalLength< MIN_LENGTH_FOR_SPLIT){
```

```
        IS_SPLIT = 0;
        TILE_NUM = 1;
    }else{
        IS_SPLIT = 1;
        TILE_NUM = DEFAULT_TILE_NUM;
    }
    context->SetBlockDim(NUM_BLOCKS);
    tiling.set_totalLength(totalLength);
    tiling.SaveToBuffer(context->GetRawTilingData()->GetData(), context-
>GetRawTilingData()->GetCapacity());
    context->GetRawTilingData()->SetDataSize(tiling.GetDataSize());
    ASCENDC_TPL_SEL_PARAM(context, D_T_X, D_T_Y, D_T_Z, TILE_NUM,
IS_SPLIT); // 设置模板参数
    size_t *currentWorkspace = context->GetWorkspaceSizes(1);
    currentWorkspace[0] = 0;
    return ge::GRAPH_SUCCESS;
}
}
```

kernel侧可以直接根据模板参数进行对应的逻辑处理：

```
#include "kernel_operator.h"
#include "add_template_custom_tiling.h"
#include "tiling_key_add_template_custom.h"

// D_T_X、D_T_Y、D_T_Z、TILE_NUM、IS_SPLIT为host侧ASCENDC_TPL_SEL_PARAM设置
的值
template <typename D_T_X, typename D_T_Y, typename D_T_Z, int TILE_NUM,
int IS_SPLIT>
 __global__ __aicore__ void add_template_custom(GM_ADDR x, GM_ADDR y,
GM_ADDR z, GM_ADDR workspace, GM_ADDR tiling)
{
    REGISTER_TILING_DEFAULT(TilingDataTemplate);
    GET_TILING_DATA_WITH_STRUCT(TilingDataTemplate, tiling_data, tiling);
    // 根据模板参数创建算子类对象
    KernelAdd<D_T_X, D_T_Y, D_T_Z> op;
    op.Init(x, y, z, tiling_data.totalLength, TILE_NUM);
    // 根据模板参数选择处理逻辑。
    if (IS_SPLIT == 0) {
        op.Process1();
    } else if (IS_SPLIT == 1) {
        op.Process2();
    }
}
```

这样我们就通过Tiling模板编程实现了tilingkey相同的效果。

---

6. 开发使用workspace、属性、Tilingkey的算子

下面我们将根据本节课的内容完成一个带属性，实现时使用workspace以及tilingkey模板的简单算子(这里重点讲解本节课知识使用，算子实现未做泛化，仅支持[8, 2048]输入)，算子原型为：

算子类型(OpType)	Clamp				
算子输入	**name**	**shape**	**data type**	**format**	**default**
	x	[8, 2048]	int32, float	ND, ND	/
算子输出	y	[8, 2048]	int32, float	ND, ND	/
算子属性	min	/	float	/	0

6.1 算子原型Json文件创建

根据上文算子原型，我们可以得知，要实现的算子为Clamp，支持Shape为[8, 2048]的int32或float32类型数据运算，且带有一个float类型的属性，默认为0。执行下面代码完成算子原型Json创建。

In [ ]:
```
%%writefile Sources/03.05/op.json
[{
    "op": "Clamp",
    "input_desc": [{
            "name": "x",
            "param_type": "required",
            "format": ["ND", "ND"],
            "type": ["int32", "float"]
        }
    ],
    "output_desc": [{
        "name": "y",
        "param_type": "required",
        "format": ["ND", "ND"],
        "type": ["int32", "float"]
    }],
    "attr":[
        {
            "name":"min",
            "type":"float",
            "param_type": "optional",
            "default_value":0
        }
    ]
}]
```

6.2 算子工程创建

完成算子原型json创建后，使用msopgen工具根据json文件创建算子工程：

In [ ]:
```
# 清除已有的custom_op目录
!rm -rf Sources/03.05/custom_op

# 使用已有的算子原型json文件创建算子工程
!msopgen gen -i Sources/03.05/op.json -c ai_core-ascend910b1 -lan cpp -out Sources/
```

命令执行完后，会创建算子工程，目录结构如下所示：

```
custom_op
|--- framework
|--- op_host
|   |--- clamp.cpp          // 包含算子原型注册、tiling实现 shape与
Dtype推导内容
|   |--- CMakeLists.txt                  // host侧CMakeLists文件，
一般不需要修改
|--- op_kernel
|   |--- clamp_tiling.h     // 算子tiling定义文件
|   |--- clamp.cpp          // 算子代码实现文件
|   |--- CMakeLists.txt                  // kernel侧CMakeLists文
件，一般不需要修改
|--- CMakeLists.txt                      // 根目录CMakeLists文件，
一般不需要修改
|--- CMakePresets.json                   // CMake编译配置文件
|--- build.sh                            // 编译脚本
```

工程目录中的op_kernel和op_host包含了算子的核心实现文件。op_kernel下存放kernel侧算子实现。op_host下存放host侧代码实现，包括算子原型定义、host侧tiling实现。

- **op_host/clamp.cpp**: 文件名会根据op.json内定义的算子名生成，包含算子原型注册、tiling实现 Shape与Dtype推导内容。
- **op_kernel/clamp_tiling.h**: 文件名会根据op.json内定义的算子名生成，包含算子tiling结构体定义。
- **op_kernel/clamp.cpp**: 文件名会根据op.json内定义的算子名生成，包含算子代码实现。
- **CMakePresets.json**: CMake编译配置文件，一般只需要修改ASCEND_CANN_PACKAGE_PATH为实际CANN安装路径。

执行以下命令查看刚刚生成的算子工程目录结构：

In [ ]:
```
!cd Sources/03.05/custom_op;find . -maxdepth 2 -print | sed -e 's;[^/]*/;|____;g;s;
```

6.3 算子Tiling模板文件添加

这里我们在op_kernel目录下创建模板参数和模板参数组合文件，命名为tiling_key_clamp.h。由于我们包含了int32/float两种数据类型，那么我们可以定义输入和输出的模板参数D_T_X、D_T_Y支持 C_DT_INT32, C_DT_FLOAT。并设置输入和输出的模板参数D_T_X、D_T_Y类型相同时才是合法的组合。

In [ ]:
```
%%writefile  Sources/03.05/custom_op/op_kernel/tiling_key_clamp.h

#ifndef TILING_KEY_CLAMP_H
#define TILING_KEY_CLAMP_H
#include "ascendc/host_api/tiling/template_argument.h"

ASCENDC_TPL_ARGS_DECL(Clamp, // 算子名
ASCENDC_TPL_DATATYPE_DECL(D_T_X, C_DT_INT32, C_DT_FLOAT, ASCENDC_TPL_INPUT(0)),
ASCENDC_TPL_DATATYPE_DECL(D_T_Y, C_DT_INT32, C_DT_FLOAT, ASCENDC_TPL_OUTPUT(0)),
);

ASCENDC_TPL_SEL(
    ASCENDC_TPL_ARGS_SEL(
    ASCENDC_TPL_DATATYPE_SEL(D_T_X, C_DT_INT32),
    ASCENDC_TPL_DATATYPE_SEL(D_T_Y, C_DT_INT32),
    ),
    ASCENDC_TPL_ARGS_SEL(
    ASCENDC_TPL_DATATYPE_SEL(D_T_X, C_DT_FLOAT),
    ASCENDC_TPL_DATATYPE_SEL(D_T_Y, C_DT_FLOAT),
    ),
);
#endif  // TILING_KEY_CLAMP_H
```

执行以下命令查看添加模板文件后的的算子工程目录结构：

In [ ]:
```
!cd Sources/03.05/custom_op;find . -maxdepth 2 -print | sed -e 's;[^/]*/;|____;g;s;
```

6.4 Tiling结构体定义

修改op_kernel目录下的clamp_tiling.h文件，完成Tiling结构体定义。本章节不重点讲解泛化实现，因此Tiling结构体只需定义切分次数和总数据长度两个核心字段即可。这里我们额外增加一个min字段，用于存放属性min，该属性会由TilingData带入kernel中参与后续计算。

In [ ]:
```
%%writefile  Sources/03.05/custom_op/op_kernel/clamp_tiling.h
#ifndef CLAMP_TILING_H
#define CLAMP_TILING_H
#include <cstdint>

struct ClampTilingData {
    uint32_t totalLength;
    uint32_t tileNum;
    float min;
};
#endif
```

6.5 Host侧实现

修改op_host目录下的clamp.cpp文件，完成host侧实现。相较于传统的Tiling函数实现方式，使用编程模板配置TilingKey时，需额外完成以下两步核心操作：

1. 引入依赖：首先需导入包含模板定义的头文件tiling_key_clamp.h；

2. 参数赋值：在Tiling函数的实现体内，通过ASCENDC_TPL_SEL_PARAM宏完成模板参数的赋值。

此外，因当前算子包含自定义属性，需在Tiling实现函数内先获取这些属性，并通过TilingData传递至kernel中参与运算；同时，由于本场景需要使用workspace，还需在Tiling函数内提前配置好计划使用的workspace内存大小。

In [ ]:
```
%%writefile  Sources/03.05/custom_op/op_host/clamp.cpp
#include "../op_kernel/clamp_tiling.h"
#include "register/op_def_registry.h"
#include "../op_kernel/tiling_key_clamp.h"
#include "tiling/platform/platform_ascendc.h"

namespace optiling {
static ge::graphStatus TilingFunc(gert::TilingContext* context)
{
    auto ascendcPlatform = platform_ascendc::PlatformAscendC(context->GetPlatformIn
    ClampTilingData *tiling = context->GetTilingData<ClampTilingData>();
    uint32_t totalLength = context->GetInputShape(0)->GetOriginShape().GetShapeSize
    // 获取数据类型
    ge::DataType dtype_x = context->GetInputDesc(0)->GetDataType();
    ge::DataType dtype_y = context->GetOutputDesc(0)->GetDataType();
    uint32_t D_T_X = static_cast<int>(dtype_x),  D_T_Y = static_cast<int>(dtype_y);

    // 设置Tiling数据
    float min_value = *context->GetAttrs()->GetFloat(0);
    tiling->totalLength = totalLength;
    tiling->tileNum = 8;
    tiling->min = min_value;
    context->SetBlockDim(8);
    // 配置模板参数
    ASCENDC_TPL_SEL_PARAM(context, D_T_X, D_T_Y);

    /*
    测试用例为[8, 2048],每个核分到2048个数据运算，核内切8次，每次运算256个数据，这里假设我
    */
    size_t userWorkspaceSize = 256 * 4;
    size_t systemWorkspaceSize = static_cast<size_t>(ascendcPlatform.GetLibApiWorkS
    size_t *currentWorkspace = context->GetWorkspaceSizes(1);
    currentWorkspace[0] = userWorkspaceSize + systemWorkspaceSize;
  return ge::GRAPH_SUCCESS;
}
}

namespace ge {
static ge::graphStatus InferShape(gert::InferShapeContext* context)
{
    const gert::Shape* x1_shape = context->GetInputShape(0);
    gert::Shape* y_shape = context->GetOutputShape(0);
    *y_shape = *x1_shape;
    return GRAPH_SUCCESS;
}
static ge::graphStatus InferDataType(gert::InferDataTypeContext *context)
{
```

```
    const auto inputDataType = context->GetInputDataType(0);
    context->SetOutputDataType(0, inputDataType);
    return ge::GRAPH_SUCCESS;
}
}

namespace ops {
class Clamp : public OpDef {
public:
    explicit Clamp(const char* name) : OpDef(name)
    {
        this->Input("x")
            .ParamType(REQUIRED)
            .DataType({ge::DT_INT32, ge::DT_FLOAT})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND})
            .UnknownShapeFormat({ge::FORMAT_ND, ge::FORMAT_ND});
        this->Output("y")
            .ParamType(REQUIRED)
            .DataType({ge::DT_INT32, ge::DT_FLOAT})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND})
            .UnknownShapeFormat({ge::FORMAT_ND, ge::FORMAT_ND});
        this->Attr("min").Float();
        this->SetInferShape(ge::InferShape).SetInferDataType(ge::InferDataType);
        this->AICore()
            .SetTiling(optiling::TilingFunc);
        this->AICore().AddConfig("ascend910b");

    }
};

OP_ADD(Clamp);
}
```

6.6 Kernel侧实现

修改op_kernel目录下的clamp.cpp文件，完成Kernel侧代码实现。相较于传统的TilingKey实现方式，使用编程模板配置TilingKey时，需额外完成以下两步核心操作：

1. 引入依赖：首先需导入包含模板定义的头文件tiling_key_clamp.h；
2. 模板参数获取：在核函数上添加模板参数，用于接收host侧设置的模板参数，并且去掉核函数的extern "C"修饰。
3. 核函数内根据模板参数判断算子执行逻辑。

此外，因当前算子包含自定义属性，需通过TilingData获取属性并参与运算；算子本身支持int32、float32两种类型，这里我们按照float类型直接使用Maxs计算获取结果，int32类型我们先转为float类型保存到workspace的缓存中，再从workspace中获取float类型的值使用Maxs计算。整体计算流程如下：



In [ ]:

```
%%writefile  Sources/03.05/custom_op/op_kernel/clamp.cpp

#include "kernel_operator.h"
#include "clamp_tiling.h"
#include "tiling_key_clamp.h"
#include "kernel_operator_dump_tensor_intf_impl.h"
constexpr int32_t BUFFER_NUM = 1;
constexpr int32_t QUEUE_DEPTH = 1;

template <class dtypeX, class dtypeY>
class KernelClamp {
public:
    __aicore__ inline KernelClamp() {}
    __aicore__ inline void Init(GM_ADDR x, GM_ADDR y, GM_ADDR workspace, uint32_t t
    {
        this->blockLength = totalLength / AscendC::GetBlockNum();
        this->tileNum = tileNum;
        this->min = min;
        this->tileLength = this->blockLength / tileNum / BUFFER_NUM;
        xGm.SetGlobalBuffer((__gm__ dtypeX *)x + this->blockLength * AscendC::GetBl
        yGm.SetGlobalBuffer((__gm__ dtypeY *)y + this->blockLength * AscendC::GetBl
        tmpGm.SetGlobalBuffer((__gm__ float *)workspace);
        pipe.InitBuffer(inQueueX, BUFFER_NUM, this->tileLength * sizeof(dtypeX));
        pipe.InitBuffer(outQueueY, BUFFER_NUM, this->tileLength * sizeof(dtypeY));
        pipe.InitBuffer(outQueueTmp, BUFFER_NUM, this->tileLength * sizeof(float));
    }

    __aicore__ inline void Process()
    {
        int32_t loopCount = this->tileNum * BUFFER_NUM;
        for (int32_t i = 0; i < loopCount; i++) {
            CopyIn(i);
            Compute(i);
            CopyOut(i);
        }
        AscendC::printf("Dtype float, Core %ld executed %d times in total\n",  Asce
    }

    __aicore__ inline void Process2()
    {
        int32_t loopCount = this->tileNum * BUFFER_NUM;
        for (int32_t i = 0; i < loopCount; i++) {
            CopyIn(i);
            Compute2(i);
            CopyOut(i);
        }
        AscendC::printf("Dtype int32, Core %ld executed %d times in total\n",  Asce
    }

private:
    __aicore__ inline void CopyIn(int32_t progress)
    {
        AscendC::LocalTensor<dtypeX> xLocal = inQueueX.AllocTensor<dtypeX>();
        AscendC::DataCopy(xLocal, xGm[progress * this->tileLength], this->tileLengt
        inQueueX.EnQue(xLocal);
```

```
    }
    __aicore__ inline void Compute(int32_t progress)
    {
        AscendC::LocalTensor<dtypeX> xLocal = inQueueX.DeQue<dtypeX>();
        AscendC::LocalTensor<dtypeY> yLocal = outQueueY.AllocTensor<dtypeY>();
        AscendC::Maxs(yLocal, xLocal, this->min, this->tileLength);
        outQueueY.EnQue<dtypeY>(yLocal);
        inQueueX.FreeTensor(xLocal);
    }

    __aicore__ inline void Compute2(int32_t progress)
    {
        AscendC::LocalTensor<dtypeX> xLocal = inQueueX.DeQue<dtypeX>();
        AscendC::LocalTensor<dtypeY> yLocal = outQueueY.AllocTensor<dtypeY>();
        // 这里假设我们把输入先转成float，然后暂存于workspace; ps: 实际上并不需要，这里只
        AscendC::LocalTensor<float> tmpLocal =  outQueueTmp.AllocTensor<float>();
        AscendC::Cast(tmpLocal, xLocal, AscendC::RoundMode::CAST_RINT, this->tileLe
        AscendC::DataCopy(tmpGm, tmpLocal, this->tileLength);
        // 然后从workspace中取出缓存的float类型数据，再和float类型min计算
        int32_t eventIDMTE3_MTE2 = static_cast<int32_t>(GetTPipePtr()->FetchEventID
        AscendC::SetFlag<AscendC::HardEvent::MTE3_MTE2>(eventIDMTE3_MTE2);
        AscendC::WaitFlag<AscendC::HardEvent::MTE3_MTE2>(eventIDMTE3_MTE2);
        AscendC::DataCopy(tmpLocal, tmpGm, this->tileLength);
        int32_t eventIDMTE2_V = static_cast<int32_t>(GetTPipePtr()->FetchEventID(As
        AscendC::SetFlag<AscendC::HardEvent::MTE2_V>(eventIDMTE2_V);
        AscendC::WaitFlag<AscendC::HardEvent::MTE2_V>(eventIDMTE2_V);
        AscendC::Maxs(tmpLocal, tmpLocal, this->min, this->tileLength);
        AscendC::Cast(yLocal, tmpLocal, AscendC::RoundMode::CAST_RINT, this->tileLe
        outQueueTmp.FreeTensor(tmpLocal);
        outQueueY.EnQue<dtypeY>(yLocal);
        inQueueX.FreeTensor(xLocal);
    }
    __aicore__ inline void CopyOut(int32_t progress)
    {
        AscendC::LocalTensor<dtypeY> yLocal = outQueueY.DeQue<dtypeY>();
        AscendC::DataCopy(yGm[progress * this->tileLength], yLocal, this->tileLengt
        outQueueY.FreeTensor(yLocal);
    }

private:
    AscendC::TPipe pipe;
    AscendC::TQue<AscendC::TPosition::VECIN, QUEUE_DEPTH> inQueueX;
    AscendC::TQue<AscendC::TPosition::VECOUT, QUEUE_DEPTH> outQueueY;
    AscendC::TQue<AscendC::TPosition::VECOUT, QUEUE_DEPTH> outQueueTmp;
    AscendC::GlobalTensor<dtypeX> xGm;
    AscendC::GlobalTensor<dtypeY> yGm;
    AscendC::GlobalTensor<float> tmpGm;
    uint32_t blockLength;
    uint32_t tileNum;
    uint32_t tileLength;
    float min;
};

template <typename D_T_X, typename D_T_Y>
 __global__ __aicore__ void clamp(GM_ADDR x, GM_ADDR y, GM_ADDR workspace, GM_ADDR
```

```
    REGISTER_TILING_DEFAULT(ClampTilingData);
    GET_TILING_DATA_WITH_STRUCT(ClampTilingData, tilingData, tiling);
    KernelClamp<D_T_X, D_T_Y> op;
    op.Init(x, y ,workspace,  tilingData.totalLength, tilingData.tileNum,tilingData

    if constexpr (std::is_same_v<D_T_X, float> && std::is_same_v<D_T_Y, float>) {
        op.Process();
    } else {
        op.Process2();
    }
}
```

6.7 调用代码说明

当前算子包含自定义属性，因此在调用二段式接口的第一段接口时，需显式传入属性值。本示例中设置属性min = 3，由于所有输入值均为-2，经算子计算后，预期输出结果为 3。

In [ ]:

```
%%writefile Sources/03.05/aclnn_test.cpp

#include <algorithm>
#include <cstdint>
#include <iostream>
#include <vector>

#include "acl/acl.h"
#include "aclnn_clamp.h"

#define SUCCESS 0
#define FAILED 1

#define CHECK_RET(cond, return_expr) \
    do {                             \
        if (!(cond)) {               \
            return_expr;             \
        }                            \
    } while (0)

#define LOG_PRINT(message, ...)         \
    do {                                \
        printf(message, ##__VA_ARGS__); \
    } while (0)

int64_t GetShapeSize(const std::vector<int64_t> &shape)
{
    int64_t shapeSize = 1;
    for (auto i : shape) {
        shapeSize *= i;
    }
    return shapeSize;
}

int Init(int32_t deviceId, aclrtStream *stream)
{
    // Fixed code, acl initialization
```

```
    auto ret = aclInit(nullptr);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclInit failed. ERROR: %d\n", ret); re
    ret = aclrtSetDevice(deviceId);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtSetDevice failed. ERROR: %d\n", r
    ret = aclrtCreateStream(stream);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtCreateStream failed. ERROR: %d\n"

    return SUCCESS;
}

template <typename T>
int CreateAclTensor(const std::vector<T> &hostData, const std::vector<int64_t> &sha
                    aclDataType dataType, aclTensor **tensor)
{
    auto size = GetShapeSize(shape) * sizeof(T);
    // Call aclrtMalloc to allocate device memory
    auto ret = aclrtMalloc(deviceAddr, size, ACL_MEM_MALLOC_HUGE_FIRST);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtMalloc failed. ERROR: %d\n", ret)

    // Call aclrtMemcpy to copy host data to device memory
    ret = aclrtMemcpy(*deviceAddr, size, hostData.data(), size, ACL_MEMCPY_HOST_TO_
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtMemcpy failed. ERROR: %d\n", ret)

    // Call aclCreateTensor to create a aclTensor object
    *tensor = aclCreateTensor(shape.data(), shape.size(), dataType, nullptr, 0, acl
                              shape.size(), *deviceAddr);
    return SUCCESS;
}

void DestroyResources(std::vector<void *> tensors, std::vector<void *> deviceAddrs,
                      int32_t deviceId, void *workspaceAddr = nullptr)
{
    // Release aclTensor and device
    for (uint32_t i = 0; i < tensors.size(); i++) {
        if (tensors[i] != nullptr) {
            aclDestroyTensor(reinterpret_cast<aclTensor *>(tensors[i]));
        }
        if (deviceAddrs[i] != nullptr) {
            aclrtFree(deviceAddrs[i]);
        }
    }
    if (workspaceAddr != nullptr) {
        aclrtFree(workspaceAddr);
    }
    // Destroy stream and reset device
    aclrtDestroyStream(stream);
    aclrtResetDevice(deviceId);
    aclFinalize();
}

int main(int argc, char **argv)
{
    // 1. (Fixed code) Initialize device / stream, refer to the list of external in
    // Update deviceId to your own device id
    int32_t deviceId = 0;
    aclrtStream stream;
```

```
    auto ret = Init(deviceId, &stream);
    CHECK_RET(ret == 0, LOG_PRINT("Init acl failed. ERROR: %d\n", ret); return FAIL

    // 2. Create input and output, need to customize according to the interface of
    std::vector<int64_t> inputXShape = {8, 2048};
    std::vector<int64_t> outputYShape = {8, 2048};
    double min = 3.0;
    void *inputXDeviceAddr = nullptr;
    void *inputYDeviceAddr = nullptr;
    void *outputYDeviceAddr = nullptr;
    aclTensor *inputX = nullptr;

    aclTensor *outputY = nullptr;
    std::vector<int32_t> inputXHostData(inputXShape[0] * inputXShape[1]);
    std::vector<int32_t> outputYHostData(outputYShape[0] * outputYShape[1]);
    for (int i = 0; i < inputXShape[0] * inputXShape[1]; ++i) {
        inputXHostData[i] = -2;
        outputYHostData[i] = 0;
    }
    std::vector<void *> tensors = {inputX, outputY};
    std::vector<void *> deviceAddrs = {inputXDeviceAddr, outputYDeviceAddr};
    // Create inputX aclTensor
    ret = CreateAclTensor(inputXHostData, inputXShape, &inputXDeviceAddr, aclDataTy
    CHECK_RET(ret == ACL_SUCCESS, DestroyResources(tensors, deviceAddrs, stream, de
    // Create outputY aclTensor
    ret = CreateAclTensor(outputYHostData, outputYShape, &outputYDeviceAddr, aclDat
    CHECK_RET(ret == ACL_SUCCESS, DestroyResources(tensors, deviceAddrs, stream, de

    // 3. Call the API of the custom operator library
    uint64_t workspaceSize = 0;
    aclOpExecutor *executor;
    // Calculate the workspace size and allocate memory for it
    ret = aclnnClampGetWorkspaceSize(inputX, min, outputY, &workspaceSize, &executo
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclnnClampGetWorkspaceSize failed. ERR
              DestroyResources(tensors, deviceAddrs, stream, deviceId); return FAIL

    void *workspaceAddr = nullptr;
    if (workspaceSize > 0) {
        ret = aclrtMalloc(&workspaceAddr, workspaceSize, ACL_MEM_MALLOC_HUGE_FIRST)
        CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("allocate workspace failed. ERROR:
                  DestroyResources(tensors, deviceAddrs, stream, deviceId, workspac
    }
    // Execute the custom operator
    ret = aclnnClamp(workspaceAddr, workspaceSize, executor, stream);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclnnClamp failed. ERROR: %d\n", ret);
              DestroyResources(tensors, deviceAddrs, stream, deviceId, workspaceAdd

    // 4. (Fixed code) Synchronize and wait for the task to complete
    ret = aclrtSynchronizeStream(stream);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtSynchronizeStream failed. ERROR:
              DestroyResources(tensors, deviceAddrs, stream, deviceId, workspaceAdd

    // 5. Get the output value, copy the result from device memory to host memory,
    // interface of the API
    auto size = GetShapeSize(outputYShape);
    std::vector<int32_t> resultData(size, 0);
```

```
    ret = aclrtMemcpy(resultData.data(), resultData.size() * sizeof(resultData[0]),
                      size * sizeof(int32_t), ACL_MEMCPY_DEVICE_TO_HOST);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("copy result from device to host failed
              DestroyResources(tensors, deviceAddrs, stream, deviceId, workspaceAdd

    // 6. Detroy resources, need to modify according to the interface of the API
    DestroyResources(tensors, deviceAddrs, stream, deviceId, workspaceAddr);

    // print the output result
    std::vector<int32_t> goldenData(size, 3);

    LOG_PRINT("result is:\n");
    for (int64_t i = 0; i < 10; i++) {
        LOG_PRINT("%d ", resultData[i]);
    }
    LOG_PRINT("\n");
    if (std::equal(resultData.begin(), resultData.end(), goldenData.begin())) {
        LOG_PRINT("test pass\n");
    } else {
        LOG_PRINT("test failed\n");
        return FAILED;
    }
    return SUCCESS;
}
```

执行调用测试：

In [ ]:
```
# 编译部署算子
!cd Sources/03.05/custom_op;bash build.sh;./build_out/custom_opp*.run --install-pat
# 部署算子后编译调用代码
!g++ -I$ASCEND_TOOLKIT_HOME/include -I${HOME}/vendors/customize/op_api/include -L$A
# 设置自定义算子so路径并执行调用代码
!source ${HOME}/vendors/customize/bin/set_env.bash;./Sources/03.05/execute_op
```

预期输出包含类似内容：

```
result is:
3 3 3 3 3 3 3 3 3 3
test pass
```

课后实践

请根据本节课内容，给AddCustomTemplate算子添加max/min属性，并确保AddCustomTemplate结果不会大于max,不会小于min,

$$AddCustomTemplate(x, y, min, max) = \begin{cases} max & \text{if } sum > max \\ min & \text{if } sum < min \\ sum & \text{otherwise} \end{cases}$$

算子原型及需要支持的shape如下：

算子类型(OpType)	AddCustomTemplate				
	name	**shape**	**data type**	**format**	**value**
算子输入	x	[8, 2048]	int8, float	ND, ND	/
	y	[8, 2048]	int8, float	ND, ND	/
算子输出	z	[8, 2048]	int8, float	ND, ND	/
算子属性	max	/	float	/	0.0
	min	/	float	/	0.0

实现时不需要考虑int8类型取值范围限制，x、y取值均小于60。

算子原型json文件定义

算子原型json文件已提供，不需要修改。

In [ ]:
```
%%writefile Sources/03.05/add_custom_template.json

[{
    "op": "AddCustomTemplate",
    "input_desc": [{
            "name": "x",
            "param_type": "required",
            "format": ["ND", "ND"],
            "type": ["int8", "float"]
        },
        {
            "name": "y",
            "param_type": "required",
            "format": ["ND", "ND"],
            "type": ["int8", "float"]
        }
    ],
    "output_desc": [{
        "name": "z",
        "param_type": "required",
        "format": ["ND", "ND"],
        "type": ["int8", "float"]
    }],
    "attr":[
        {
            "name":"max",
```

```
            "type":"float",
            "param_type": "optional",
            "default_value":0
        },
        {
            "name":"min",
            "type":"float",
            "param_type": "optional",
            "default_value":0
        }
    ]
}]
```

创建算子工程

In [ ]:
```
# 清除已有的custom_op目录
!rm -rf Sources/03.05/custom_op
!msopgen gen -i Sources/03.05/add_custom_template.json -c ai_core-ascend910b1 -lan
```

添加模板定义

这里已给出一份基础的模板定义，请根据需要修改。

In [ ]:
```
%%writefile  Sources/03.05/custom_op/op_kernel/tiling_key_add_custom_template.h

#ifndef TILING_KEY_ADD_CUSTOM_TEMPLATE_H
#define TILING_KEY_ADD_CUSTOM_TEMPLATE_H
#include "ascendc/host_api/tiling/template_argument.h"

ASCENDC_TPL_ARGS_DECL(AddCustomTemplate, // 算子名
ASCENDC_TPL_DATATYPE_DECL(D_T_X, C_DT_INT8, C_DT_FLOAT, ASCENDC_TPL_INPUT(0)),
ASCENDC_TPL_DATATYPE_DECL(D_T_Y, C_DT_INT8, C_DT_FLOAT, ASCENDC_TPL_INPUT(1)),
);

ASCENDC_TPL_SEL(
    ASCENDC_TPL_ARGS_SEL(
    ASCENDC_TPL_DATATYPE_SEL(D_T_X, C_DT_INT8),
    ASCENDC_TPL_DATATYPE_SEL(D_T_Y, C_DT_INT8),
    ),
    ASCENDC_TPL_ARGS_SEL(
    ASCENDC_TPL_DATATYPE_SEL(D_T_X, C_DT_FLOAT),
    ASCENDC_TPL_DATATYPE_SEL(D_T_Y, C_DT_FLOAT),
    ),
);
#endif
```

完成Host侧实现

In [ ]:
```
%%writefile Sources/03.05/custom_op/op_host/add_custom_template.cpp
#include "../op_kernel/add_custom_template_tiling.h"
#include "register/op_def_registry.h"
#include "../op_kernel/tiling_key_add_custom_template.h"

namespace optiling {
static ge::graphStatus TilingFunc(gert::TilingContext* context)
```

```
{

  AddCustomTemplateTilingData *tiling = context->GetTilingData<AddCustomTemplateTil
  const gert::StorageShape* x1_shape = context->GetInputShape(0);
  int32_t data_sz = 1;
  for (int i = 0; i < x1_shape->GetStorageShape().GetDimNum(); i++)
    data_sz *= x1_shape->GetStorageShape().GetDim(i);
  tiling->size = data_sz;
  context->SetBlockDim(8);
  size_t *currentWorkspace = context->GetWorkspaceSizes(1);
  currentWorkspace[0] = 0;
  return ge::GRAPH_SUCCESS;
}
}

namespace ge {
static ge::graphStatus InferShape(gert::InferShapeContext* context)
{
    const gert::Shape* x1_shape = context->GetInputShape(0);
    gert::Shape* y_shape = context->GetOutputShape(0);
    *y_shape = *x1_shape;
    return GRAPH_SUCCESS;
}
static ge::graphStatus InferDataType(gert::InferDataTypeContext *context)
{
    const auto inputDataType = context->GetInputDataType(0);
    context->SetOutputDataType(0, inputDataType);
    return ge::GRAPH_SUCCESS;
}
}

namespace ops {
class AddCustomTemplate : public OpDef {
public:
    explicit AddCustomTemplate(const char* name) : OpDef(name)
    {
        this->Input("x")
            .ParamType(REQUIRED)
            .DataType({ge::DT_INT8, ge::DT_FLOAT})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND})
            .UnknownShapeFormat({ge::FORMAT_ND, ge::FORMAT_ND});
        this->Input("y")
            .ParamType(REQUIRED)
            .DataType({ge::DT_INT8, ge::DT_FLOAT})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND})
            .UnknownShapeFormat({ge::FORMAT_ND, ge::FORMAT_ND});
        this->Output("z")
            .ParamType(REQUIRED)
            .DataType({ge::DT_INT8, ge::DT_FLOAT})
            .Format({ge::FORMAT_ND, ge::FORMAT_ND})
            .UnknownShapeFormat({ge::FORMAT_ND, ge::FORMAT_ND});
        this->Attr("max").AttrType(OPTIONAL).Float(0);
        this->Attr("min").AttrType(OPTIONAL).Float(0);
        this->SetInferShape(ge::InferShape).SetInferDataType(ge::InferDataType);
        this->AICore()
            .SetTiling(optiling::TilingFunc);
```

```
        this->AICore().AddConfig("ascend910b");
    }
};

OP_ADD(AddCustomTemplate);
}
```

完成tiling 结构体定义

请根据需求修改Tiling结构体

In [ ]:
```
%%writefile  Sources/03.05/custom_op/op_kernel/add_custom_template_tiling.h
#ifndef ADD_CUSTOM_TEMPLATE_TILING_H
#define ADD_CUSTOM_TEMPLATE_TILING_H
#include <cstdint>

struct AddCustomTemplateTilingData {
    uint32_t size;
};
#endif
```

完成kernel 侧实现

In [ ]:
```
%%writefile  Sources/03.05/custom_op/op_kernel/add_custom_template.cpp
#include "kernel_operator.h"
#include "add_custom_template_tiling.h"
#include "tiling_key_add_custom_template.h"

template <typename D_T_X, typename D_T_Y>
__global__ __aicore__ void add_custom_template(GM_ADDR x, GM_ADDR y, GM_ADDR z, GM_
    REGISTER_TILING_DEFAULT(AddCustomTemplateTilingData);
    GET_TILING_DATA(tilingData, tiling);
    // TODO: user kernel impl
}
```

尝试修改下面的单算子API调用代码，将其修改成输出结果在[8,16]之间, 数据类型为int8。

In [ ]:
```
%%writefile Sources/03.05/aclnn_test.cpp
#include <algorithm>
#include <cstdint>
#include <iostream>
#include <vector>
#include "acl/acl.h"
#include "aclnn_add_custom_template.h"

#define SUCCESS 0
#define FAILED 1

#define CHECK_RET(cond, return_expr) \
    do {                             \
        if (!(cond)) {               \
            return_expr;             \
        }                            \
    } while (0)
```

```
#define LOG_PRINT(message, ...)         \
    do {                                \
        printf(message, ##__VA_ARGS__); \
    } while (0)

int64_t GetShapeSize(const std::vector<int64_t> &shape)
{
    int64_t shapeSize = 1;
    for (auto i : shape) {
        shapeSize *= i;
    }
    return shapeSize;
}

int Init(int32_t deviceId, aclrtStream *stream)
{
    // Fixed code, acl initialization
    auto ret = aclInit(nullptr);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclInit failed. ERROR: %d\n", ret); re
    ret = aclrtSetDevice(deviceId);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtSetDevice failed. ERROR: %d\n", r
    ret = aclrtCreateStream(stream);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtCreateStream failed. ERROR: %d\n"

    return SUCCESS;
}

template <typename T>
int CreateAclTensor(const std::vector<T> &hostData, const std::vector<int64_t> &sha
                    aclDataType dataType, aclTensor **tensor)
{
    auto size = GetShapeSize(shape) * sizeof(T);
    // Call aclrtMalloc to allocate device memory
    auto ret = aclrtMalloc(deviceAddr, size, ACL_MEM_MALLOC_HUGE_FIRST);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtMalloc failed. ERROR: %d\n", ret)

    // Call aclrtMemcpy to copy host data to device memory
    ret = aclrtMemcpy(*deviceAddr, size, hostData.data(), size, ACL_MEMCPY_HOST_TO_
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtMemcpy failed. ERROR: %d\n", ret)

    // Call aclCreateTensor to create a aclTensor object
    *tensor = aclCreateTensor(shape.data(), shape.size(), dataType, nullptr, 0, acl
                              shape.size(), *deviceAddr);
    return SUCCESS;
}

void DestroyResources(std::vector<void *> tensors, std::vector<void *> deviceAddrs,
                      int32_t deviceId, void *workspaceAddr = nullptr)
{
    // Release aclTensor and device
    for (uint32_t i = 0; i < tensors.size(); i++) {
        if (tensors[i] != nullptr) {
            aclDestroyTensor(reinterpret_cast<aclTensor *>(tensors[i]));
        }
        if (deviceAddrs[i] != nullptr) {
            aclrtFree(deviceAddrs[i]);
```

```
        }
    }
    if (workspaceAddr != nullptr) {
        aclrtFree(workspaceAddr);
    }
    // Destroy stream and reset device
    aclrtDestroyStream(stream);
    aclrtResetDevice(deviceId);
    aclFinalize();
}

int main(int argc, char **argv)
{
    // 1. (Fixed code) Initialize device / stream, refer to the list of external in
    // Update deviceId to your own device id
    int32_t deviceId = 0;
    aclrtStream stream;
    auto ret = Init(deviceId, &stream);
    CHECK_RET(ret == 0, LOG_PRINT("Init acl failed. ERROR: %d\n", ret); return FAIL

    // 2. Create input and output, need to customize according to the interface of
    std::vector<int64_t> inputXShape = {8, 2048};
    std::vector<int64_t> inputYShape = {8, 2048};
    std::vector<int64_t> outputZShape = {8, 2048};
    double max = 2;
    double min = 1;
    void *inputXDeviceAddr = nullptr;
    void *inputYDeviceAddr = nullptr;
    void *outputZDeviceAddr = nullptr;
    aclTensor *inputX = nullptr;
    aclTensor *inputY = nullptr;
    aclTensor *outputZ = nullptr;
    std::vector<int8_t> inputXHostData(inputXShape[0] * inputXShape[1]);
    std::vector<int8_t> inputYHostData(inputYShape[0] * inputYShape[1]);
    std::vector<int8_t> outputZHostData(outputZShape[0] * outputZShape[1]);
    for (int i = 0; i < inputXShape[0] * inputXShape[1]; ++i) {
        inputXHostData[i] = 1;
        inputYHostData[i] = 2;
        outputZHostData[i] = 0;
    }
    std::vector<void *> tensors = {inputX, inputY, outputZ};
    std::vector<void *> deviceAddrs = {inputXDeviceAddr, inputYDeviceAddr, outputZD
    // Create inputX aclTensor
    ret = CreateAclTensor(inputXHostData, inputXShape, &inputXDeviceAddr, aclDataTy
    CHECK_RET(ret == ACL_SUCCESS, DestroyResources(tensors, deviceAddrs, stream, de
    // Create inputY aclTensor
    ret = CreateAclTensor(inputYHostData, inputYShape, &inputYDeviceAddr, aclDataTy
    CHECK_RET(ret == ACL_SUCCESS, DestroyResources(tensors, deviceAddrs, stream, de
    // Create outputZ aclTensor
    ret = CreateAclTensor(outputZHostData, outputZShape, &outputZDeviceAddr, aclDat
    CHECK_RET(ret == ACL_SUCCESS, DestroyResources(tensors, deviceAddrs, stream, de

    // 3. Call the API of the custom operator library
    uint64_t workspaceSize = 0;
    aclOpExecutor *executor;
    // Calculate the workspace size and allocate memory for it
```

```
    ret = aclnnAddCustomTemplateGetWorkspaceSize(inputX, inputY,max, min, outputZ,
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclnnAddCustomTemplateGetWorkspaceSize
              DestroyResources(tensors, deviceAddrs, stream, deviceId); return FAIL

    void *workspaceAddr = nullptr;
    if (workspaceSize > 0) {
        ret = aclrtMalloc(&workspaceAddr, workspaceSize, ACL_MEM_MALLOC_HUGE_FIRST)
        CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("allocate workspace failed. ERROR:
                  DestroyResources(tensors, deviceAddrs, stream, deviceId, workspac
    }
    // Execute the custom operator
    ret = aclnnAddCustomTemplate(workspaceAddr, workspaceSize, executor, stream);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclnnAdd failed. ERROR: %d\n", ret);
              DestroyResources(tensors, deviceAddrs, stream, deviceId, workspaceAdd

    // 4. (Fixed code) Synchronize and wait for the task to complete
    ret = aclrtSynchronizeStream(stream);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("aclrtSynchronizeStream failed. ERROR:
              DestroyResources(tensors, deviceAddrs, stream, deviceId, workspaceAdd

    // 5. Get the output value, copy the result from device memory to host memory,
    // interface of the API
    auto size = GetShapeSize(outputZShape);
    std::vector<int8_t> resultData(size, 0);
    ret = aclrtMemcpy(resultData.data(), resultData.size() * sizeof(resultData[0]),
                      size * sizeof(int8_t), ACL_MEMCPY_DEVICE_TO_HOST);
    CHECK_RET(ret == ACL_SUCCESS, LOG_PRINT("copy result from device to host failed
              DestroyResources(tensors, deviceAddrs, stream, deviceId, workspaceAdd

    // 6. Detroy resources, need to modify according to the interface of the API
    DestroyResources(tensors, deviceAddrs, stream, deviceId, workspaceAddr);

    // print the output result
    std::vector<int8_t> goldenData(size, 2);

    LOG_PRINT("result is:\n");
    for (int64_t i = 0; i < 10; i++) {
        LOG_PRINT("%d ", resultData[i]);
    }
    LOG_PRINT("\n");
    if (std::equal(resultData.begin(), resultData.end(), goldenData.begin())) {
        LOG_PRINT("test pass\n");
    } else {
        LOG_PRINT("test failed\n");
        return FAILED;
    }
    return SUCCESS;
}
```

运行测试

In [ ]:

```
# 编译算子工程并部署编译出的算子包
!cd Sources/03.05/custom_op;bash build.sh;./build_out/custom_opp*.run --install-pat

# 部署算子后编译调用代码
```

```
!g++ -I$ASCEND_TOOLKIT_HOME/include -I${HOME}/vendors/customize/op_api/include -L$A
# 设置自定义算子so路径并执行调用代码
!source ${HOME}/vendors/customize/bin/set_env.bash;./Sources/03.05/execute_add_op
```

查看答案

实现方式不唯一，这里给出的答案仅供参考。

host侧实现：

In [ ]:
```
!cat ./answer/03.05_answer/op_host/add_custom_template.cpp
```

Tiling结构体定义：

In [ ]:
```
!cat ./answer/03.05_answer/op_kernel/add_custom_template_tiling.h
```

TilingKey模板定义：

In [ ]:
```
!cat ./answer/03.05_answer/op_kernel/tiling_key_add_custom_template.h
```

Kernel实现：

In [ ]:
```
!cat answer/03.05_answer/op_kernel/add_custom_template.cpp
```